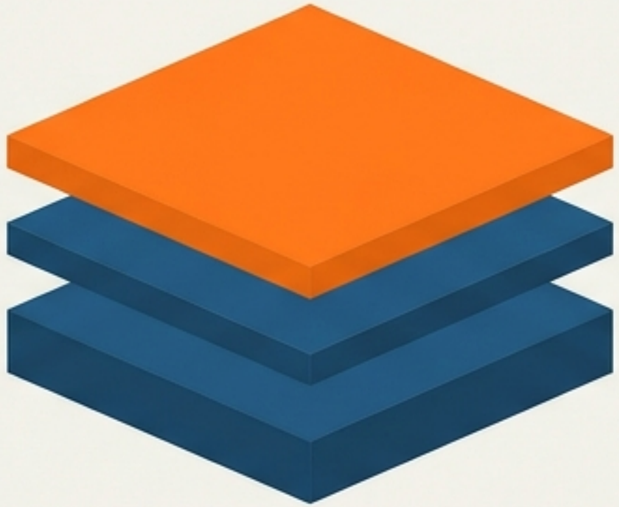
The Stack Data Structure

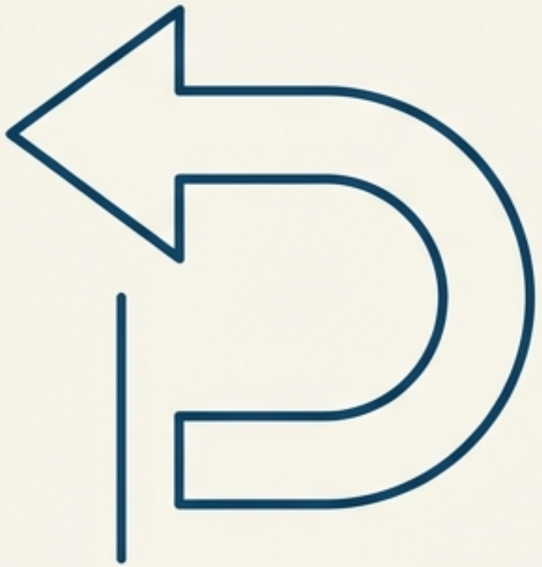
A Python Implementation Guide

Understanding LIFO, Recursion Foundations, and Class Construction.



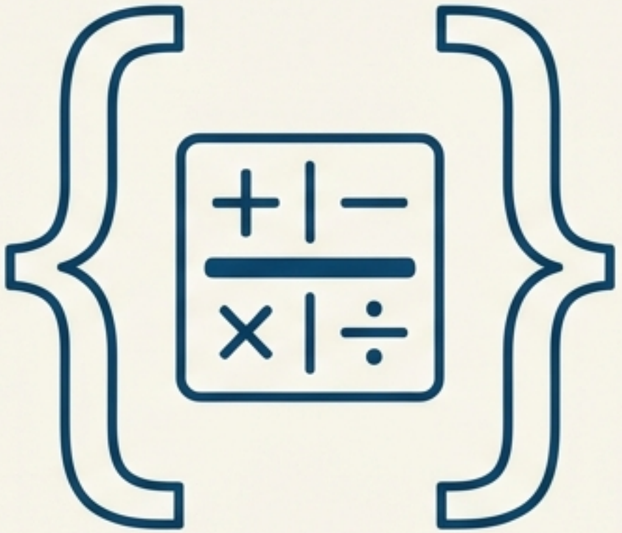
`ver 1.0 // structure: LIFO`

More Than Just Theory



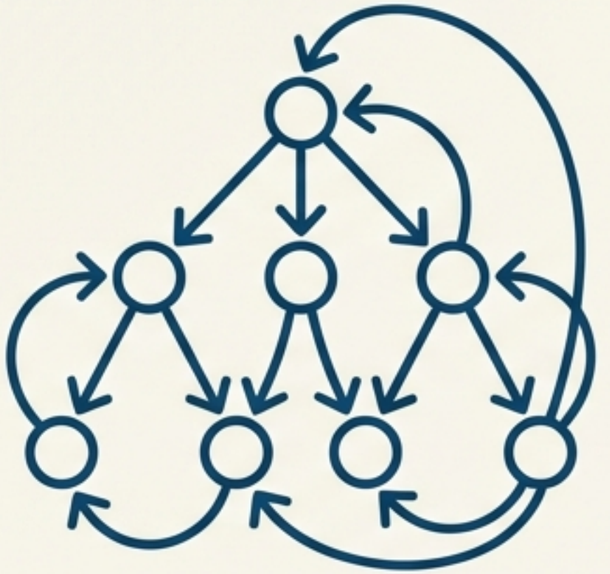
Undo/Redo Operations

The backbone of text editors and browser navigation.



Expression Evaluation

Parsing math and code syntax correctness.



Recursion Foundation

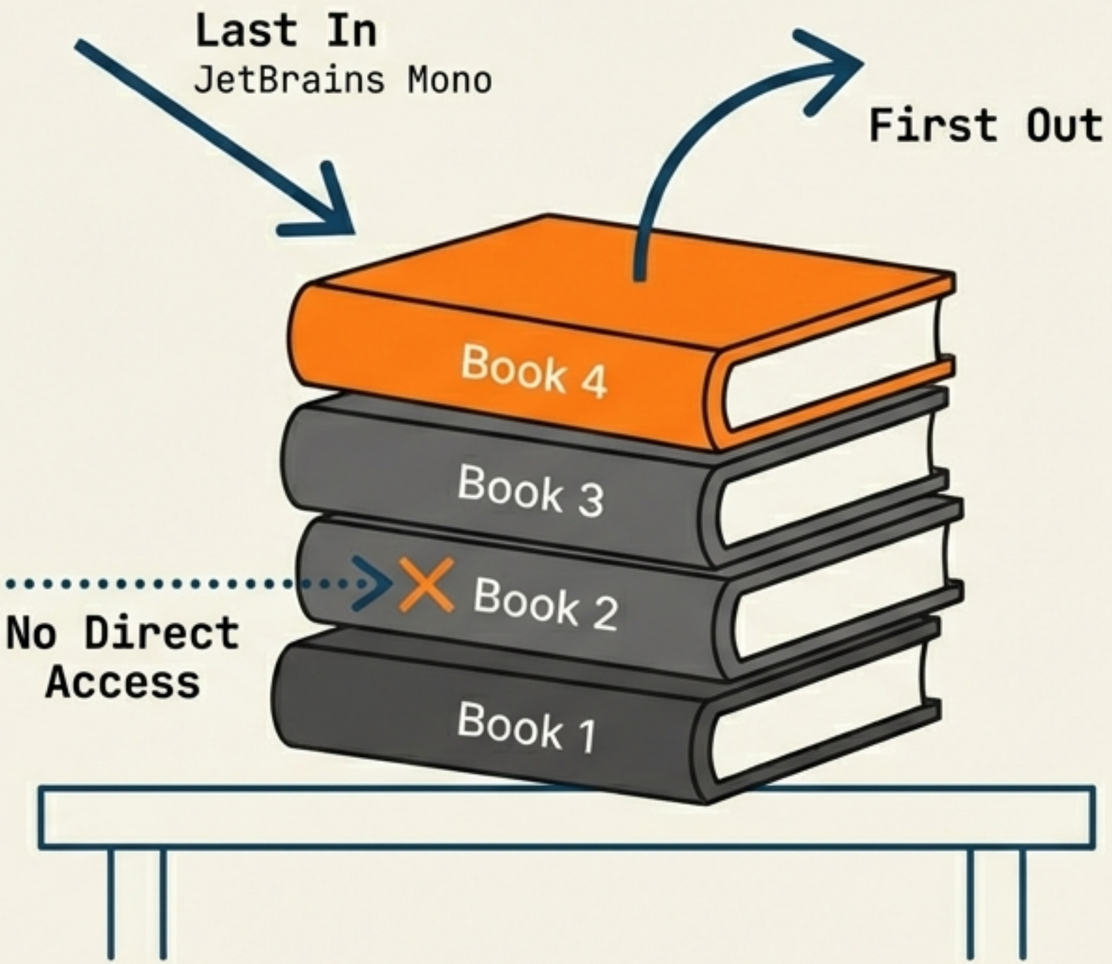
Managing memory allocation for function calls.

Stacks are the hidden engine of order in computing.

The LIFO Principle

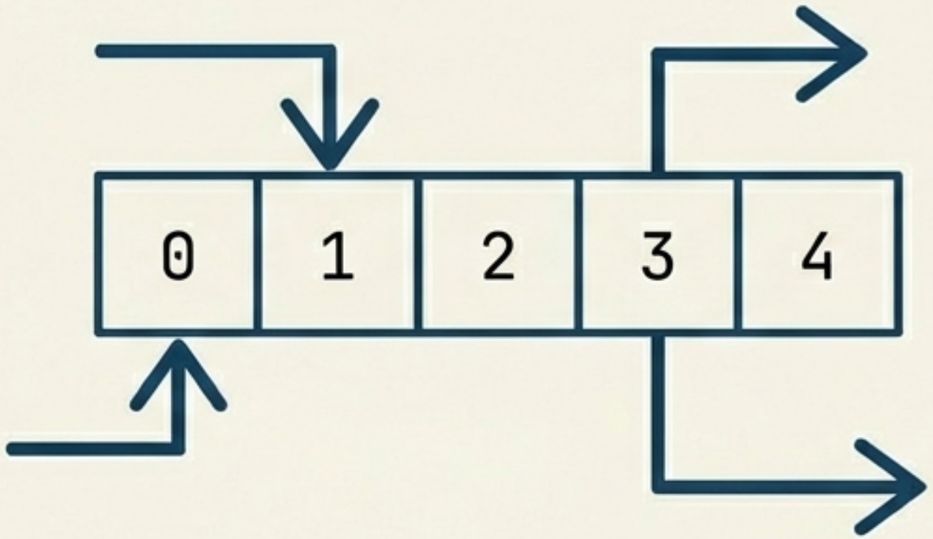
Last In, First Out

Imagine a physical stack of books. You cannot pull a book from the middle without disturbing the pile. You must remove the top item before accessing the one beneath it.



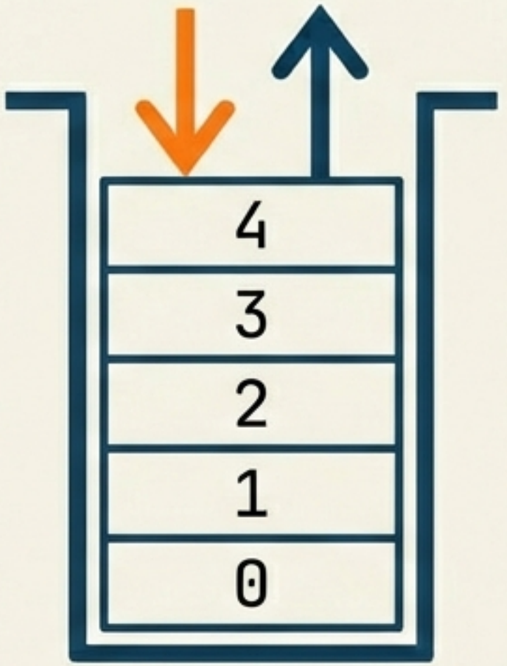
Restriction is a Feature

Standard List



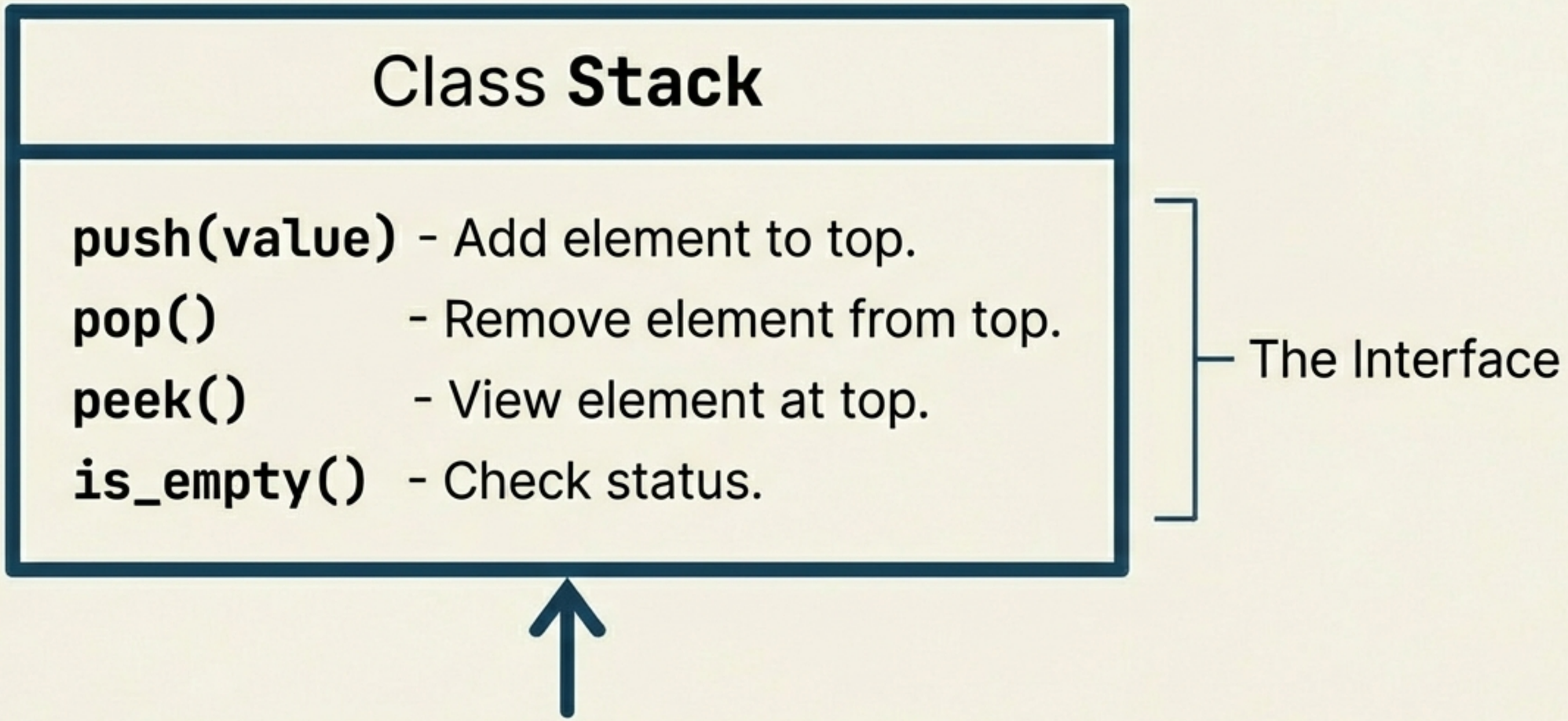
Random Access = Chaos.
Vulnerable to accidental data corruption.

The Stack



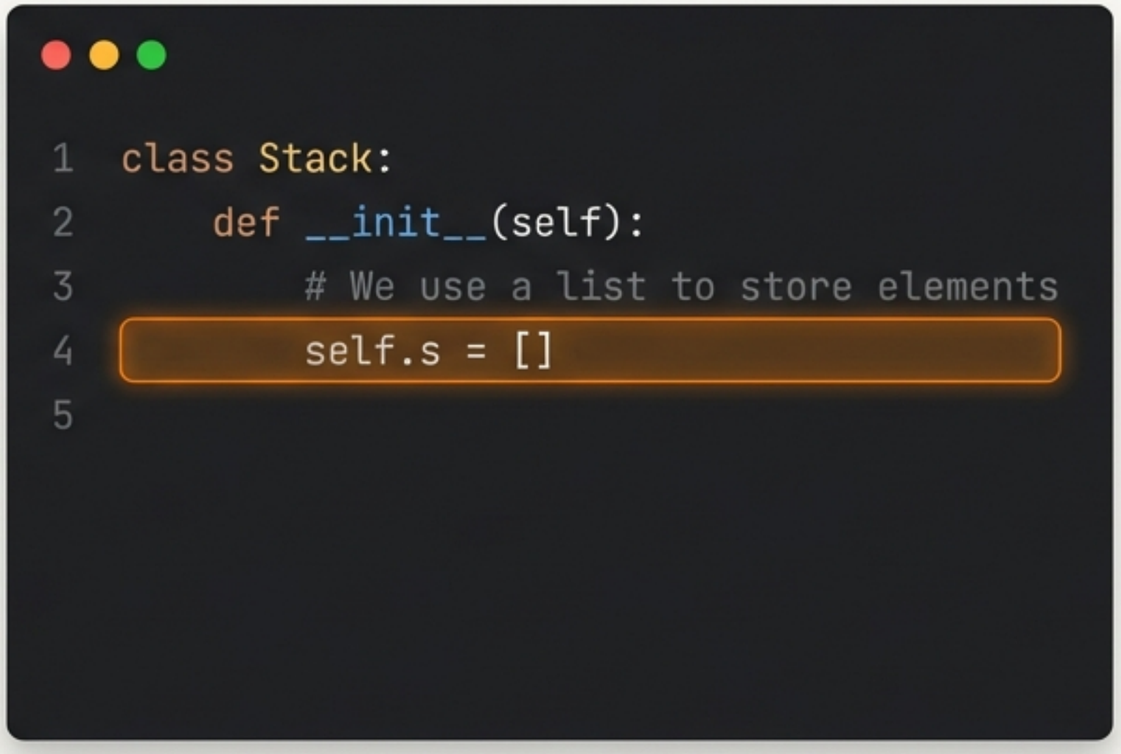
Restricted Access = Discipline.
Forces strict data flow logic.

The Blueprint: Stack API



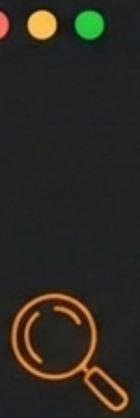
Step 1: The Foundation

We initialize the class with an empty list. Unlike C++ or Java Java arrays, Python lists are dynamic. We do not need to pre-define the size or worry about memory overflow during initialization.



Step 2: Utility Methods

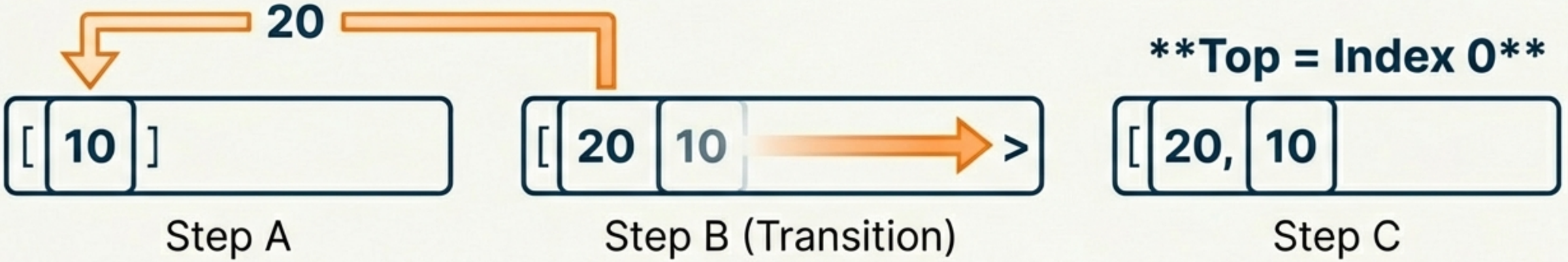
Basic housekeeping methods wrap standard list functionality. **is_empty()** is a critical guard rail—it prevents us from trying to remove items that don't exist.



```
def length(self):
    return len(self.s)

def is_empty(self):
    return len(self.s) == 0
```

Step 3: The 'Push' Strategy




```
1     def push(self, value):
2         # Insert at Index 0 (The Top)
3         self.s.insert(0, value)
```

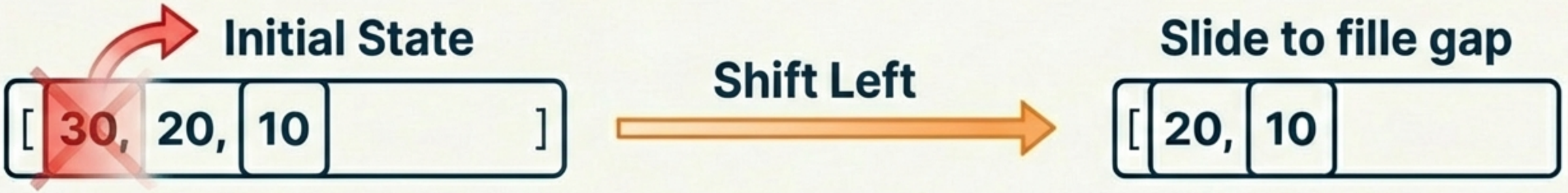
New items are inserted at the start, shifting previous elements down.

Step 4: Peek (Inspection)

Look, don’t touch. Peek allows us to read the value at the Top (Index 0) without removing it. We must include a safety check for empty stacks.

```
def peek(self):
    if not self.is_empty():
        return self.s[0]
    else:
        raise Exception('Stack is empty')
```

Step 5: Pop (Removal)



We must pop from Index 0 to maintain LIFO consistency.

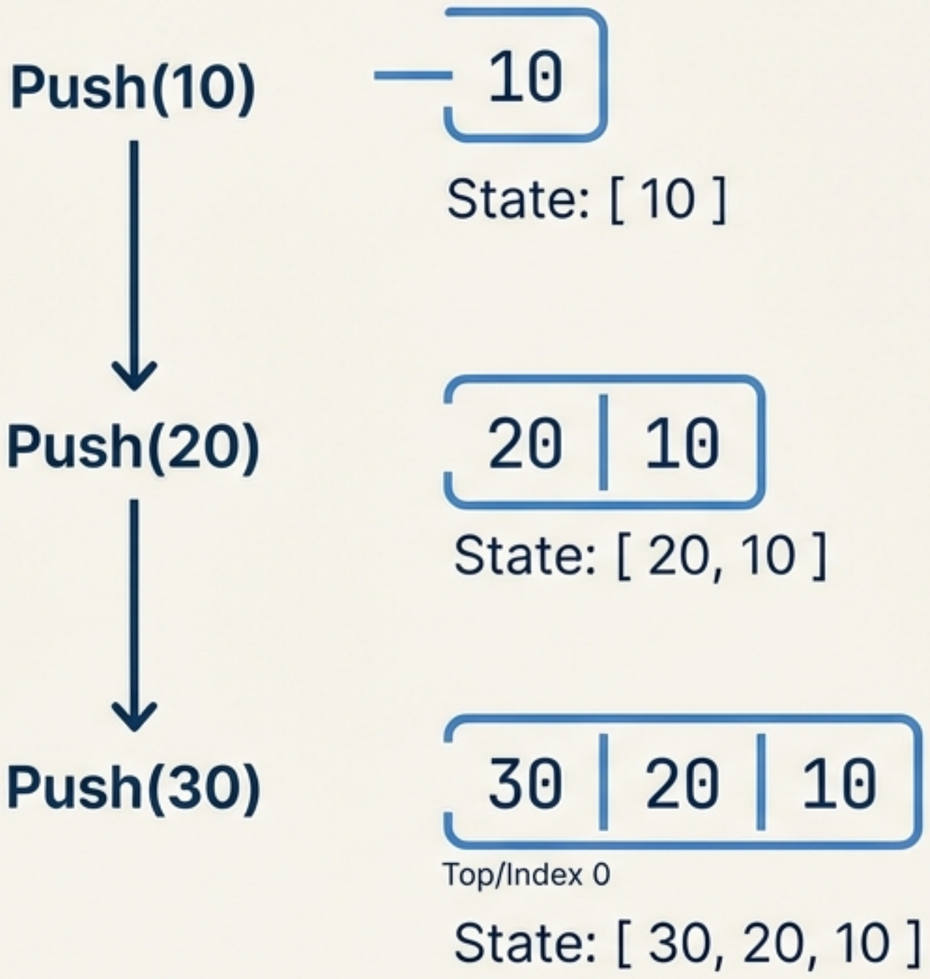
```
def pop(self):
    if not self.is_empty():
        return self.s.pop(0)
    else:
        raise Exception('Stack is empty')
```

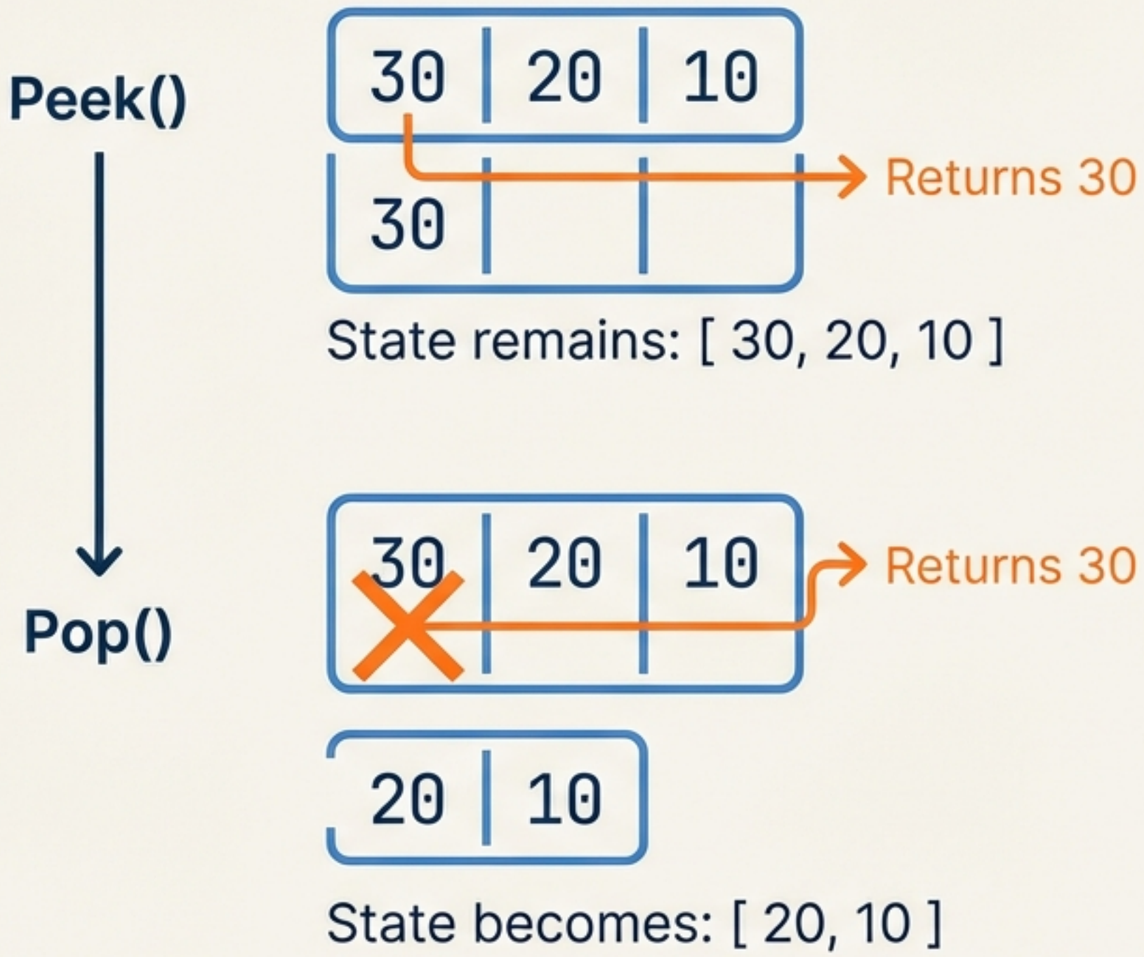
Guarding Against Errors

“If someone has no money, you can’t borrow from them. If the stack is empty, you can’t delete from it.”

Using `raise Exception` prevents silent failures and crashes.

Execution Trace





Optimization & Homework

Method A: The Tutorial Way	Method B: The Optimized Way
Top = Index 0 Uses `insert(0)` and `pop(0)` Performance: **O(n)** (Slow, requires shifting)	Top = Last Index (-1) Uses `append()` and `pop()` Performance: **O(1)** (Fast, no shifting)

Key Takeaways

- **LIFO Rule:** Last In, First Out is the governing principle.
- **Abstraction:** We wrap a simple Python List to enforce disciplined access.
- **Index Strategy:** While we used Index 0 as Top, using the End of the list is faster.
- **Safety First:** Always check `is_empty()` before accessing data.

Continue the Journey

Complete the optimization homework using `append()`.
Share your code on Twitter.

Based on the 'Stack in Python' lecture by Prateek // Chai aur Code